PS 4 HACKATHON BUILD BIBLE

Can I Afford It Next Month?

The complete execution document for Problem Statement 4: every hard ban and must-have as a checklist, the LLM + code architecture, tech stack decisions, the forecast and verdict spec, a 24-hour timeline, the demo script, and the judge defence.

Prepared for the PS 4 Team

Compliance · Architecture · Execution · Defence

OCTOBER 2026 · FINAL · RESEARCH FROZEN

Table of Contents

1. PS 4 Decoded: What the Problem Statement Demands 1

2. Hard Bans - Jo Bilkul Nahi Karna 2

3. Must-Haves - Jo Karna Hi Hai 3

4. Architecture: The LLM Boundary 4

5. Tech Stack Decision (Python vs Node/TS) 5

6. Data Ingestion: CSV, PDF, Multi-Account 6

7. Categorization Engine 7

8. Dedup, Transfers, Failed UPI 8

9. Forecast + Affordability Engine (The Heart) 8

10. Anomaly Detection (MAD, Rules-First) 11

11. Gemini Integration, API + DB Design 11

12. Unit Tests + Definition of Done 13

13. The 24-Hour Timeline 14

14. Demo Plan: Personas, Traps, Script 14

15. Judge Q&A; Defence + Verified Sources 15

1. PS 4 Decoded: What the Problem Statement Demands

> **TL;DR (team version, Hinglish)**
> Ye document tumhara single source of truth hai. Part 1 me wo sab hai jo PS 4 me **bilkul mana** hai (Chapter 2) aur jo **karna hi mandatory** hai (Chapter 3). Part 2 pura build plan hai: architecture, tech stack, har engine ka spec, API, DB, tests. Part 3 me 24 ghante ki timeline, demo script aur judges ke sawalon ke ready jawab. Ek line me architecture: **LLM samajhta hai, code saabit karta hai.** Ab koi naya research mat kholna - yahi final plan hai.

Problem Statement 4 asks for one thing in plain words: a tool that reads bank statements - CSV and PDF exports, from more than one account - and answers a single question the user types in: **“Can I afford ₹X next month?”** The answer must be a verdict the user can trust, and the problem statement adds one hard engineering constraint that shapes the whole build: **all calculations must be done in code, not generated by the language model.** That single sentence is not a restriction to work around; it is the winning architecture, and Chapter 4 shows exactly how to exploit it.

Read as engineers, the problem statement makes five demands. Each demand maps to a concrete, testable property of your build, and each one is something a judge can verify in under a minute. If any row of the table below is missing from your demo, you are leaving points on the table - so treat this table as the definition of “done” for the MVP.

PS 4 requirement	What it means in the build	How you prove it live
Reads bank statements (CSV + PDF)	Upload endpoint; per-bank CSV adapters; PDF text extraction with a graceful failure path	Upload 2 CSVs from 2 different banks + 1 PDF during the demo
Multiple accounts	Account_id on every transaction; merge, dedup and transfer-pair logic across accounts	Show two accounts where the same salary transfer appears once and is asked about
Answers “Can I afford ₹X next month?”	Question parsed to JSON by the LLM; forecast + verdict computed by pure code functions	Type a fresh price the team did not rehearse; verdict returns in seconds
Fully transparent, explainable	Every answer ships with a receipt object: inputs, formula, result, data range used	Open the receipt panel on screen; every number traces back to named transactions
Calculations in code, not the LLM	LLM only parses and phrases; arithmetic lives in unit-tested functions	Open the test suite; run one test live; show that the LLM receives numbers as text

Judges at hackathons rarely grade the idea - every team gets the same problem statement. They grade four things: does the demo **work live**, can they **follow the calculation** when you walk through it, does the tool **admit uncertainty honestly** instead of hallucinating confidence, and is the scope **restrained enough to be real**. The frozen decisions in this document are all tuned to those four criteria. One more criterion that teams forget: talk to two real people about how they actually decide affordability before the event - quoting a real

user in the pitch (“she checks her salary date first”) scores higher than any feature.

2. Hard Bans - Jo Bilkul Nahi Karna

Every item below is a trap that has sunk real teams: it looks harmless at 2 a.m., it demos fine in your bedroom, and it dies on stage the moment a judge asks one sharp question. The third column tells you exactly how a judge catches each one. Print this table, stick it above the desk, and check it before every commit. Nothing here is a style preference - these are the rules that keep the tool honest and the demo alive.

	NEVER do this	Why it is fatal	How judges catch it
[ ]	Let the LLM do any arithmetic - even one sum or one total	Transformer multi-step arithmetic collapses as complexity grows (Dziri et al., NeurIPS 2023); the PS explicitly forbids it; one wrong rupee kills all trust	Live test: “add these three debits on screen” - if the number came from the LLM, it breaks or disagrees with the receipt
[ ]	Auto-delete or auto-merge possible duplicate transactions	Overlapping exports make duplicates *possible*, not certain; silently deleting real spend understates expenses and corrupts the forecast	“What happens if I upload the same month twice?” - upload it again and watch the ledger
[ ]	Use z-score / 3-sigma rules on fewer than ~20 points	With n = 3 months the maximum possible z-score is $(n-1)/\sqrt{n} \approx 1.15$, so a $z > 3$ rule can *never fire* - you shipped a dead rule	“Show me one anomaly this caught on 2 months of data” - it cannot, and your MAD fallback is missing
[ ]	Claim an accuracy percentage you cannot reproduce	There is no public labeled benchmark for Indian bank narrations; a fake “95%” is a credibility bomb	“Which dataset? Where is the eval?” - no answer, pitch over
[ ]	Hardcode demo numbers or outcomes into the engine	The demo works, the product does not; judges edit one CSV row and everything falls apart	“Can I change the price to 21,500?” - if the verdict does not move, it is hardcoded
[ ]	Use mean ± SD as the outlier rule	One big outlier inflates both mean and SD and hides itself (masking); Leys et al. 2013 is explicit: use median-based rules	Plant a ₹45,000 one-time credit; if it is not flagged, the rule is mean-based
[ ]	Hide the calculation behind a pretty verdict card	The PS says *transparent and explainable*; a bare YES/NO is a rejected submission	“Where does ₹18,400 of buffer come from?” - if it is not on screen, you lose the transparency criterion
[ ]	Let the LLM write the final verdict or any number in its phrasing	Phrasing LLM sees numbers as text; if it ever “helps” by rounding or recomputing, the receipt and the sentence disagree	Compare the receipt JSON with the sentence on screen; a mismatch is caught instantly
[ ]	Quote unverified statistics in the pitch (benchmark scores, DPDP fine amounts)	One caught fabrication ends the Q&A; judges remember the lie, not the product	Any judge with a phone can check the citation in 30 seconds
[ ]	Start new research or add a new library mid-hackathon	Integration risk at hour 18 is how teams end up with nothing that runs	The commit history shows a new dependency at 2 a.m. with no tests

Notice the pattern: almost every ban is a form of dishonesty - hidden math, fake accuracy, silent deletion. The problem statement is essentially grading integrity under uncertainty, which is why the honest moves (say REFUSED, show a wide band, flag duplicates and ask) are also the highest-scoring moves. Chapter 3 turns this around into the mandatory list.

3. Must-Haves - Jo Karna Hi Hai

These twelve items are the contract. Each one is small enough to build in an hour or two, and together they are exactly what a transparent affordability engine looks like. If you finish the hackathon with only this list done and nothing more, you still have a coherent, defensible product - that is the point of freezing scope.

	Always do this	Where it shows up
[ ]	Four verdict states with exact conditions: YES, TIGHT, NO, REFUSED (Chapter 9 defines each)	The verdict card; the demo's third persona exists to trigger REFUSED
[ ]	REFUSED when history is under 3 months - with the reason written on screen	New earner persona; flips to a real verdict after you add a month of data live
[ ]	Safety buffer computed in code: worst-month variable overshoot + 1 week of essentials	Receipt panel line items; judges ask “what is this buffer?” more than anything else
[ ]	Receipt object with every answer: inputs, formula, result, data range, buffer breakdown	The transparency slide and the live receipt panel
[ ]	Possible duplicates flagged and asked - never auto-deleted (Fellegi-Sunter P class)	The trap fixture demo beat with overlapping exports
[ ]	Transfer pairs between own accounts excluded from income and spend	Salary moved from Account A to B does not double-count income
[ ]	Failed UPI + reversal pairs net to zero - never counted as spend	Trap fixture; one line in the ledger view proves it
[ ]	All math in pure, unit-tested functions with a fixed random seed	Live test run during the demo; seed makes the bootstrap reproducible
[ ]	Forecast shown as a band (P10 / P50 / P90), never a single number	The band chart in the UI; the honest-uncertainty talking point
[ ]	Buffer editable by the user, verdict recomputes instantly	Judge asks “what if I keep a smaller buffer?” - you slide it down live
[ ]	Commit every 6 hours, alarm set on every phone	Missed-commit disqualification is the only way to lose without losing
[ ]	A “what we did not build” slide	It reads as maturity, and it defuses the toughest question in Chapter 15

Two of these deserve emphasis because teams skip them under time pressure. The **REFUSED path** is your differentiator: every other team will return a confident number for a user with six weeks of history, and yours will say “not enough data - here is why, here is what changes at three months.” That single behavior demonstrates the entire thesis of the problem statement. The **editable buffer** is your interactivity hook: judges remember the tool they poked. Both are cheap; build them before any polish.

4. Architecture: The LLM Boundary

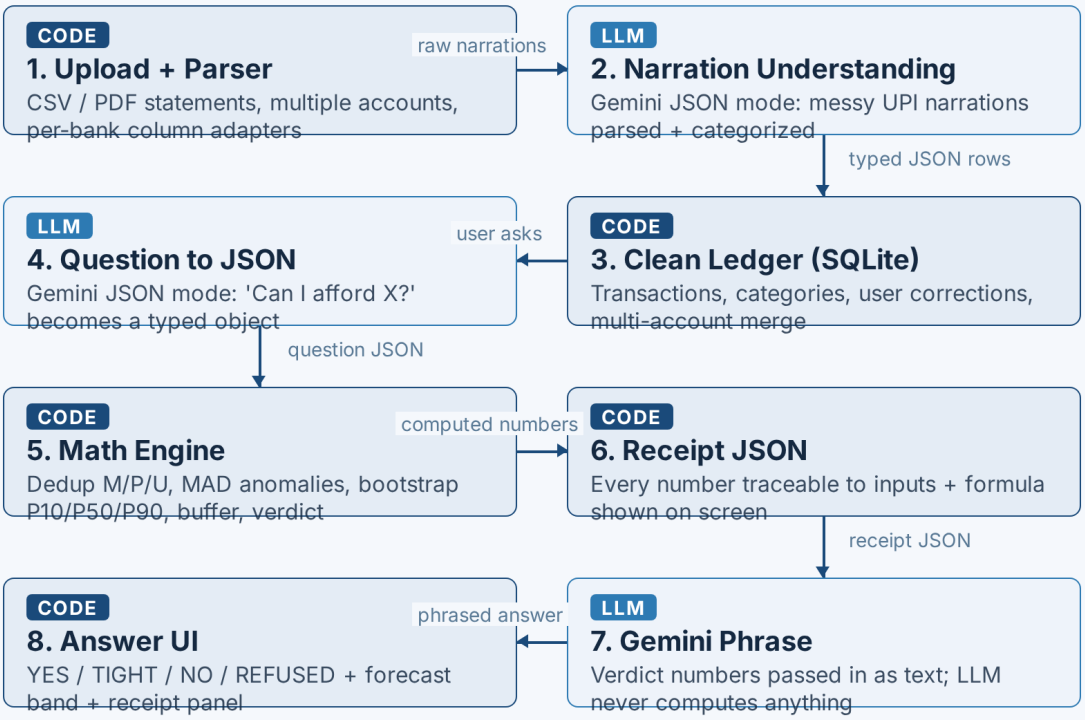


Figure 1: The PS 4 pipeline. Three LLM jobs (blue accent) are strictly bounded: understand narrations, parse the question, phrase the answer. Everything between them is deterministic, unit-tested code.

The problem statement's constraint - calculations in code, not the LLM - has a name in the research literature: the **program-aided** (PAL) split. Gao et al. (ICLR 2023) showed that when an LLM converts a problem into a program and an interpreter executes it, accuracy beats chain-of-thought by double digits on math benchmarks, because the arithmetic stops depending on the model's pattern-matching. Your build is that pattern, applied to bank statements: the LLM owns language, deterministic code owns arithmetic.

Why the LLM must never cross into arithmetic: Dziri et al. (NeurIPS 2023) showed that transformer reasoning degenerates into linearized subgraph matching - models can execute patterns they have seen, and collapse on compositions they have not. Adding a ₹20,000 price to a buffer works; “3-month median of irregular freelancer income, minus committed EMIs, plus worst-month overshoot” does not. Judges will probe exactly there. The boundary also makes the demo auditable: every number on screen was produced by a function a judge could have read on GitHub before walking up to your desk.

Job	Owner	Concrete example
Parse messy UPI narrations into typed rows	LLM	“UPI/P2A/2918374/AMAZON PAY/” + amount + date → {merchant, direction, category, confidence}
Categorize unknown merchants	LLM	“BLINKIT 8823 GGN” → groceries (sent as redacted descriptor, never with account numbers)

Job	Owner	Concrete example
Parse the user's question to JSON	LLM	“Can I afford a ₹20k laptop next month?” → {item, price_inr: 20000, horizon: next_month}
Phrase the computed verdict	LLM	Verdict object in, sentence out; numbers passed as strings, never recomputed
Every arithmetic step: medians, buffer, bootstrap, verdict, shortfall	CODE	Pure functions in one module, unit tests in CI, fixed seed, runs offline
Dedup classification, transfer detection, anomaly flags	CODE	Fellegi-Sunter M/P/U rules; MAD z-scores; deterministic and explainable

> **The one-line defence (memorize this)**
> “The LLM is the inbox, not the calculator: it reads, it classifies, it phrases - and every rupee figure is computed by a unit-tested function the judges can read. LLM samajhta hai, code saabit karta hai.”

5. Tech Stack Decision (Python vs Node/TS)

One rule is frozen before anything else: **all math lives in exactly one language.** Two stacks (say, a Python notebook for the forecast and a JS app for the demo) is how “slide numbers ≠ demo numbers” happens - the bootstrap in notebook A gets ported at 3 a.m. and quietly diverges. Pick one stack for the math engine, keep the frontend a thin client on top of it, and the receipt you show judges is produced by the same functions the tests cover.

Criterion	Python: FastAPI + pytest + pandas	Node/TS: Next API routes + Vitest
Forecast + statistics	Native: numpy quantiles, bootstrap in ~15 lines	Plain JS loops work; no scientific stack, more code to trust
CSV / data wrangling	pandas + csv module handle every bank export quirk	Hand-rolled parsing; fine for known formats, slower on surprises
PDF extraction	pdfplumber (best-in-class for statement tables)	pdf-parse works but weaker on table layouts
LLM SDK + JSON mode	google-genai Python SDK, schema-constrained output	Same capability via the REST/JS SDK
Testing story	pytest: fast, readable, runs offline	Vitest: equally good; TS types catch schema drift
Frontend coupling	Separate app (Next.js) calling FastAPI - two processes	One repo, one language end to end - fewer moving parts
Hackathon speed	Fastest for the math-heavy parts	Fastest for UI + API in a single deploy
Verdict	Pick if the team is stronger in Python or has any stats exposure	Pick if the team ships TS daily and nobody wants a second runtime

Both columns are viable; the decision is a 10-minute meeting item on team strength, not a research topic. What is **not** negotiable, whichever column wins: the frontend is **Next.js + Tailwind + shadcn/ui** (polished SaaS

look, band chart, receipt panel), the LLM is **Gemini 2.5 Flash in JSON mode** (free tier, schema-constrained output), the database is **SQLite** (zero setup, file = artifact), and an **Ollama fallback** (Llama 3.2 3B) is documented for the worst case: venue Wi-Fi dies and the cloud API is unreachable. The fallback is a 20-line adapter swap because both LLM jobs are single schema-constrained calls.

Component	Frozen choice	Why
Frontend	Next.js 15 + Tailwind + shadcn/ui	Judged on polish; band chart + receipt panel are one evening's work
LLM API	Gemini 2.5 Flash, JSON mode, temperature 0	Free tier for students; structured output mode rejects malformed JSON
Database	SQLite (single file)	No install, trivially reset between demo runs, easy to inspect on stage
LLM fallback	Ollama + Llama 3.2 3B (local)	Offline insurance; swap one base URL, same JSON contract
Math engine home	One module, pure functions, fixed seed	The single-language rule; testable without the server

6. Data Ingestion: CSV, PDF, Multi-Account

Every Indian bank exports CSV its own way: column names differ (Date/Txn Date/Value Date; Narration/Remarks/Description), amounts arrive as “1,234.50” strings with commas, debits may be a signed column or separate Dr/Cr columns, and dates flip between DD/MM/YYYY and DD-MM-YYYY. Do not build a universal parser - build a tiny **adapter config** per bank and a strict normalizer behind it. Two or three adapters are enough for the demo; the architecture is the point, not the count.

Adapter field	Example (HDFC-style)	Example (SBI-style)	Normalizer output
date_col	Date	Txn Date	ISO 8601 date
narration_col	Narration	Description	clean description string
amount_mode	signed Withdrawal/Deposit	single Dr/Cr pair	signed amount_inr (debit negative)
ref_col	Chq/Ref No	Ref No./Cheque No	ref string (dedup key part)
balance_col	Closing Balance	Balance	validated running balance
date_format	%d/%m/%Y	%d %b %Y	parsed date

- Parse dates to ISO; reject rows whose balance does not reconcile with the running balance (a free correctness check every bank gives you).
- Strip currency symbols and thousands commas; store integer paise to avoid float drift in totals.
- Collapse whitespace in narrations; keep the raw string alongside the cleaned one (receipts show the raw).
- Assign account_id from the upload; generate a stable transaction hash from (account, date, amount, ref, narration-normalized) for the dedup layer.

- Cap memory: stream the CSV; a 3,000-row file must parse in under 2 seconds on a laptop.

PDF statements: use **pdfplumber** (Python) or **pdf-parse** (Node) to extract the transaction table. Two failure modes are certain at hackathons and must be handled as features, not crashes: **password-protected** PDFs (detect, tell the user to unlock, or accept the password field) and **scanned/image** PDFs (no text layer - return a clear message and ask for CSV). Saying “scanned PDFs need OCR - we chose to fail loudly and honestly” is a better answer on stage than a silent wrong parse. Trap fixtures to prepare tonight: two overlapping CSV exports from the same account, the same salary appearing in two accounts, a 3,000-row export, a failed UPI with its reversal, and a password-protected PDF.

7. Categorization Engine

Categorization is the layer that makes forecasts meaningful (rent is committed, chai is variable), and it is also where teams over-engineer. The frozen design is a 3-layer funnel: cheap layers first, LLM only for what falls through. This keeps cost near zero, latency low, and - critically for the transparency criterion - every assignment is traceable to a rule, an alias, or an LLM call with its confidence.

Layer	What it handles	Rough share	Cost / latency
1. Alias dictionary (120-150 Indian merchants)	Exact and substring hits: Blinkit, Zomato, Swiggy, DMart, Amazon Pay, Paytm, Jio, Netflix, rent keywords (RENT, MAINT), salary keywords (SAL, NEFT CR SAL)	~60-70%	free / instant
2. Deterministic rules	Pattern classes: UPI/P2A person-to-person, card EMI tags, ATM/WDL cash, interest/charges (BANK CHARGES, GST), refunds (REFUND, RVRSL)	~15-20%	free / instant
3. LLM (Gemini, JSON mode)	Unknown free-text merchants only, sent as redacted descriptors; returns category + confidence; below threshold → 'uncategorized', user can fix	~10-25%	paise / sub-second

Layer 3 privacy rule is frozen: the request contains the merchant substring, direction and amount band - never the account number, never the full narration with references, and never the user's name. The response is validated against the schema; on any invalid JSON you retry once and then fall back to “uncategorized” rather than guessing. The user can relabel any transaction in the UI; that correction is written to the alias table, so the same merchant auto-categorizes forever after. Show this loop in the demo - it is thirty seconds long and proves the system learns.

Measurement, frozen: build a labeled set of **~200 real transactions** tonight (two people, one hour), and report **macro-F1 with n** - “macro-F1 0.84 on 200 labeled Indian narrations” is a strong, honest claim. There is no public benchmark for Indian bank narrations (verified gap during research), so nobody in the room can one-up the number, and nobody can accuse you of inventing one. Never say “95% accuracy” - say the F1 and show the confusion cases; judges trust teams that show their errors.

8. Dedup, Transfers, Failed UPI

The dedup layer uses the three-class model from classical record linkage (Fellegi and Sunter, JASA 1969): a pair of records is a **match (M)**, a **possible match (P)**, or **unmatch (U)**. The classes map directly to code behavior, and the P class is where trust is won: **flag it and ask the user - never delete silently.**

Class	Rule (frozen)	Action	Example
M	Identical transaction hash (account, date, amount, ref, normalized narration)	Auto-merge; keep one row; log it	Same CSV uploaded twice
P	Same account, same amount, dates within 3 days, near-identical narration	Flag in UI; user picks keep / duplicate; choice remembered	₹20 chai logged twice by two apps
U	Everything else	Keep both rows untouched	Two ₹200 coffees in one day

Transfers between own accounts are not income and not spend. Detection rule: an outflow in account A and an inflow in account B with |amount difference| ≤ ₹1, on the same or adjacent date. Mark both rows type = transfer, exclude them from income totals, spend totals and the forecast's net flows, and say so in the receipt (“excluded 1 internal transfer”). The demo fixture is a salary credited to Account A and moved to Account B the next day - naive tools double-count income; yours visibly does not.

Failed UPI and reversals: narrations containing FAILED / DECLINED / TEMPORARILY REVERSED with a matching reversal row within 7 days net to zero. The pair stays visible in the ledger (flagged “netted”) but contributes nothing to spend or forecast. The everyday case: a ₹20 chai paid twice where the first attempt failed and auto-reversed - a naive tool reports ₹40 of chai, yours reports ₹20. This is a one-line difference that makes judges nod.

> **Why 'flag and ask' beats 'delete and pray'**
> Auto-deletion is invisible corruption: when it is wrong, the forecast is wrong and the user never learns why. Flagging costs one UI state and buys the sentence every fintech wants: “we never touch your data without asking.”

9. Forecast + Affordability Engine (The Heart)

Everything before this chapter feeds one module. The engine takes the clean ledger and the parsed question, and returns a verdict object with a receipt. It contains exactly four computations, all pure functions: (1) daily net flows, (2) a 30-day forecast with uncertainty, (3) the safety buffer, (4) the verdict. Each is explainable on one slide, which is the real requirement - a judge should be able to re-derive the number while you watch.

9.1 Forecast: moving-block bootstrap on the user's own flows

Parametric models (ARIMA, exponential smoothing) need more history than real users have and assume stability that salaries-and-chai lives do not have. The frozen method is the **moving-block bootstrap** (Kunsch, Annals of Statistics 1989): resample the user's own recent daily net flows in contiguous blocks, preserving weekday patterns and occasional spiky days, and read the forecast distribution directly from the resamples. It is honest on thin data: where history is short, the band widens and says so - which Chapter 3 defined as a feature, and Angelopoulos and Bates (2023) formalize as conformal-style abstention: a wide interval is the tool saying “I do not know precisely.”

Parameter	Frozen value	Reason (one line)
Input series	Daily net flow (credits - debits), last 90 days	Raw, user-owned; no model assumptions to defend
Block length	30 days (contiguous)	Preserves monthly rhythm: salary, rent, weekend spikes
Draws	500 (seed = 42)	Stable quantiles; reproducible on stage
Per-draw statistic	Sum of next-30-day flows after prepending current balance	Answers the actual question: balance 30 days out
Aggregation	P10 / P50 / P90 of the 500 outcomes	Median is the plan; P10 is the honest worst case

Python reference implementation (Node/TS version: same constants, plain loop)

```
# forecast.py - pure function, no I/O, seed fixed
def bootstrap_forecast(daily_flows, opening_balance,
                       horizon_days=30, n_draws=500,
                       block_days=30, seed=42):
    """Return P10/P50/P90 of balance after `horizon_days`."""
    rng = random.Random(seed)
    n = len(daily_flows)
    if n < 90:  # caller enforces REFUSED earlier; belt and braces
        raise ValueError('need 90 days of flows')
    outcomes = []
    for _ in range(n_draws):
        start = rng.randint(0, n - block_days)
        block = daily_flows[start:start + block_days]
        outcomes.append(opening_balance + sum(block[:horizon_days]))
    outcomes.sort()
    q = lambda p: outcomes[int(p * (len(outcomes) - 1))]
    return {'p10': q(0.10), 'p50': q(0.50), 'p90': q(0.90)}
```

9.2 The safety buffer (frozen formula, user-editable)

buffer = (worst observed month's variable-spend overshoot over its median) + (7 x median daily essential spend). The first term is “how bad did a bad month get in your own history”; the second is one week of essentials so a delayed salary does not break you. Both terms are computed from the ledger, both are itemized in the receipt, and the buffer is editable in the UI with instant recompute - the judge-poke feature. The verdict compares the affordable amount against **price + buffer**, never against price alone; the buffer line is what separates a tool from a calculator.

9.3 The four verdicts (exact conditions - code these, then test them)

Verdict	Condition	On-screen explanation (LLM phrases this)
REFUSED	months_of_data < 3	“Only 2.2 months of history - a forecast would be a guess. Add one more month or upload an older statement.”
YES	P10 ≥ price + buffer	“Even in your worst-case path you clear the price plus a ₹18,400 buffer.”
TIGHT	P50 ≥ price + buffer, but P10 < price + buffer	“Affordable on the median path, short by ₹2,100 in the worst case. Delay 2 weeks or trim the buffer consciously.”
NO	P50 < price + buffer	“Median path falls ₹6,000 short - this purchase raises overdraft risk.”

9.4 The receipt object (ships with every answer)

Receipt JSON (shortened) - every screen number traces to a line here

```
{
  'question': {'item': 'laptop', 'price_inr': 20000, 'horizon': 'next_month'},
  'data_window': {'from': '2026-08-01', 'to': '2026-10-30',
                  'accounts': ['HDFC-1234', 'SBI-8890'], 'txns': 412},
  'balance_today_inr': 34210,
  'forecast': {'p10': 18500, 'p50': 24480, 'p90': 31100,
               'method': 'moving-block bootstrap',
               'draws': 500, 'block_days': 30, 'seed': 42},
  'buffer_inr': 18400,
  'buffer_parts': {'worst_month_overshoot': 14200,
                   'one_week_essentials': 4200},
  'excluded': {'internal_transfers': 1, 'netted_failed_upi': 1},
  'flags': {'possible_duplicates': 1},
  'verdict': 'TIGHT',
  'shortfall_worst_case_inr': 20000 + 18400 - 18500,
  'computed_by': 'pure functions, unit-tested, seed 42'
}
```

> **The honest-uncertainty slide line**
> A wide band on thin data is the tool telling the truth, not a bug. P50 alone is a lie; P10 is the number that keeps rent paid.

10. Anomaly Detection (MAD, Rules-First)

Anomalies matter here for one purpose: one-off spikes (a ₹45,000 chargeback credit, a ₹31,000 medical bill) must not poison the medians that income and buffer depend on. The frozen rule is the **modified z-score on the median absolute deviation** (Iglewicz and Hoaglin, 1993): $M_i = 0.6745\ (x_i - \text{median}) / \text{MAD}$, flag $|M| > 3.5$. MAD replaces mean and SD because a single extreme value inflates both and then hides itself - the masking failure Leys et al. (2013) documented across the experimental literature. Flagged transactions are visually marked and excluded from the median-income calculation (still visible in the ledger).

Guard	Rule	Why
MAD = 0 (constant series)	No outlier possible; flag nothing, say 'uniform history'	Half a month of identical subscription values must not crash the divisor
Small n (n < 10)	Report at most 1 flag, labeled 'possible'	Flagging 2 of 8 points is noise, not detection
Rule source	Rules only; no Isolation Forest in the MVP	Goldstein & Uchida (2016): no unsupervised method dominates; rules are explainable on stage
Future work slide	Isolation Forest (Liu et al., ICDM 2008)	Say it, do not build it - scope discipline is graded

The small-sample guard is not paranoia - it is arithmetic. With n = 3 points the maximum achievable standard-score is $(n - 1)/\sqrt{n} \approx 1.15$, so any $z > 3$ rule on three months of history is provably dead code. MAD-based flags with the guards above survive every probe in Chapter 15, and they demo well: point at the flagged ₹45,000 “V*PAYBCK*SPARKL” row, show the median moving when it is excluded, and the transparency criterion is paid for in ten seconds.

11. Gemini Integration, API + DB Design

Both LLM jobs are single schema-constrained calls to **Gemini 2.5 Flash** with temperature 0. Hard rules: JSON mode with an explicit schema; validate the response, retry once on failure, then fall back to rules/uncategorized - the pipeline never blocks on the LLM; and redaction - the categorizer sees merchant substrings and amount bands, never account numbers, names, or full narrations with references. Keep the two prompts and their schemas in one file with the validation code, so the Ollama fallback is a base-URL swap with the same contract.

Question-parser schema

```
// Job 1: question parser - response schema (Gemini JSON mode)
{
  'type': 'object', 'required': ['item', 'price_inr', 'horizon'],
  'properties': {
    'item':      {'type': 'string'},
    'price_inr': {'type': 'number', 'minimum': 0},
    'horizon':   {'enum': ['next_month', 'this_month', 'custom_date']},
    'custom_date': {'type': 'string'}
  },
  'additionalProperties': false
}
// Rejected parse -> one retry -> UI asks a structured follow-up.
// The LLM NEVER sees ledger data in this call.
```

Endpoint	Purpose	Returns
POST /api/upload	CSV/PDF files (multi) + account labels	parsed row counts, rejected rows
POST /api/ingest	Run categorization + dedup + transfer detection	ledger summary + flagged items
GET /api/forecast	30-day band from current ledger	P10/P50/P90 + method metadata
POST /api/question	“Can I afford...” text	verdict + receipt (single call)
POST /api/correction	relabel a transaction	updated category + alias learned
GET /api/receipt/{id}	full receipt for any past verdict	the JSON of Chapter 9.4

Table	Key columns	Purpose
accounts	id, bank, label, last4	multi-account support
transactions	id, account_id, date, amount_paise, raw_narration, clean_narration, ref, category, flags, txn_hash	the ledger; hashes feed dedup
category_aliases	pattern, category, source (dict/user), hits	correction propagation
verdicts	id, question_json, receipt_json, created_at	audit trail; /api/receipt reads here

Cost is effectively zero on the Gemini free tier (two short JSON calls per demo action). The offline story is a slide line, not a hope: “if Wi-Fi dies, we flip one environment variable to Ollama and the demo continues with the same JSON contract - the math never needed the cloud anyway.”

12. Unit Tests + Definition of Done

The test suite is not hygiene here - it is a demo prop and a trust machine. Ten focused tests, written early (Chapter 13 slots them at hour 6-10), cover every frozen rule in this document. Run one of them on stage; it lands harder than any slide. Names below are deliberately readable so judges scanning the repo get the design without reading code.

Test	Input	Expected
test_bootstrap_deterministic	same flows, seed 42, run twice	identical P10/P50/P90
test_verdict_yes_boundary	P10 exactly price + buffer	YES (boundary is inclusive)
test_verdict_tight	P50 clears, P10 short	TIGHT with shortfall = P10 - (price+buffer)
test_verdict_refused_thin	2.2 months of data	REFUSED + reason string
test_buffer_composition	3-month ledger fixture	buffer = overshoot + 7 x essentials (exact paise)
test_dup_p_never_autodeleted	overlapping exports	both rows kept, flag raised, count = 1
test_transfer_pair_excluded	A out 10000, B in 10000 next day	income and spend unchanged; type=transfer
test_failed_upi_netted	fail row + reversal row	net zero; excluded from spend
test_mad_zero_guard	8 identical values	no crash, no flags, 'uniform history'
test_amount_parsing	“1,234.50” Dr column	123450 paise, negative sign

Definition of Done (tick all before the demo slot)

- Upload 3 files from 2 banks + 1 PDF → ledger reconciles (balance check passes on every account).
- A fresh, unrehearsed question returns a verdict + receipt in under 5 seconds.
- Buffer slider edit recomputes the verdict live.
- A category correction propagates: the next identical merchant auto-categorizes.
- Overlapping-export fixture: flag raised, nothing deleted, forecast unchanged until user decides.
- pytest/vitest full suite green in one run, offline, before the demo slot.
- Git history: commits every ≤ 6 hours for the whole event.

13. The 24-Hour Timeline

Four people, four lanes (A: math engine, B: ingestion, C: frontend, D: data + demo + slides). The plan front-loads the engine because it is the part judges cannot forgive if it is fake, and back-loads rehearsal because demos are won in the last two hours. If you are a smaller team, merge lanes B and D - never merge A and C.

Window	Lane A (math)	Lane B (ingestion)	Lane C (frontend)	Lane D (data/demo)
H0-2	repo + skeleton + test harness	adapter config format	Next.js + shadcn scaffold	fixtures spec written
H2-6	net flows + buffer functions	CSV parsers + normalizer	upload page + ledger table	build 3 persona CSVs
H6-10	bootstrap + verdict + tests green	PDF path + failure modes	question box + verdict card	label 200 narrations
H10-14	MAD flags + receipt object	dedup M/P/U + transfers	receipt panel + band chart	trap fixtures CSVs
H14-20	support lane C; performance	correction propagation API	wire APIs; buffer slider	SLEEP (rotating 5h)
H20-22	freeze engine; tag v1	freeze; tag v1	polish + empty/error states	slides + ‘not built’ slide
H22-24	standby for live fixes	standby	demo run-through x2	dry run + Q&A; drill

- Commit alarm on every phone at H6 / H12 / H18 - the only unbuilt feature that can disqualify you.
- Cut-line order if behind: what-if simulator → Isolation Forest → multi-PDF batch upload. Never cut: unit tests, receipt, REFUSED path.
- The H14-20 sleep slot is not optional; a 4 a.m. bootstrap bug costs more than the hour it stole.
- By H20 the engine is frozen: after that, only UI strings and demo data change - nothing that can alter a verdict.

14. Demo Plan: Personas, Traps, Script

The demo is a story in six beats, carried by three personas chosen to expose the three things that make this tool different: the band (uncertainty), the flags (never touch data silently), and the refusal (honesty on thin data). Rehearse twice at H22-24 with a team-mate playing the hostile judge - the one who edits the price and re-uploads a CSV.

Persona	History	Income shape	Expected verdict	Story beat
Rohan, salaried	14 months	₹72k monthly, stable	YES (comfortable band)	Clean case; show receipt and buffer parts
Meera, freelancer	8 months	₹40k-150k, lumpy	TIGHT (wide band)	Uncertainty is the feature; show P10 vs P50
Arjun, first job	2.2 months	₹45k, new	REFUSED → flip to real verdict	Add a month live → verdict appears; honesty wins

Trap fixture	What it proves	Expected behavior
Overlapping CSV exports	dedup P-class discipline	flag + ask; nothing deleted; forecast stable
Salary moved A → B	transfer exclusion	income not double-counted; receipt shows exclusion
Failed UPI + reversal	netting logic	₹20 chai, not ₹40
₹20 chai logged twice	P-class ask, not auto-delete	user decides; choice remembered
Password-protected PDF	graceful failure	clear message + CSV path offered
Scanned PDF (image)	graceful failure	"no text layer - upload CSV"; no crash
₹45,000 odd credit	MAD anomaly	flagged; median income excludes it; explained

Six beats (total 4 minutes)

1. 0:00 - Cold open on Arjun (2.2 months): type the question, show REFUSED with the reason. One sentence: "most tools will bluff here."
2. 0:40 - Add a month of data live, re-ask: verdict appears with band + receipt. "Same math every time."
3. 1:20 - Meera (freelancer): wide band; open the receipt; point at buffer parts and the excluded transfer.
4. 2:10 - Hostile-judge beat: re-upload overlapping export → flag + ask; relabel one merchant → correction propagates.
5. 2:50 - Slide: architecture (Figure 1) + "calculations in code" - run one unit test live.
6. 3:30 - Slide: what we did not build + backtest numbers (band coverage, P50 MAE) + macro-F1 with n. Close on the one-liner.

> **Three numbers to have on the last slide**
> macro-F1 with n (own labeled set) - backtest band coverage + P50 MAE (rolling-origin, Hyndman fpp3 method) - commit count at every 6h alarm. Numbers you can defend beat features you cannot.

15. Judge Q&A; Defence + Verified Sources

Ten questions cover 95% of what judges ask. Answers are one or two sentences plus a pointer - the pointer (a test, a receipt line, a slide) is what makes them land. Rehearse these out loud in the H22-24 dry run.

Q1. Why use an LLM at all if calculations must be code?

A. The PS bans LLM calculation, not LLM understanding. We run the PAL architecture (Gao et al., ICLR 2023): the LLM parses narrations and questions to JSON and phrases results; every rupee is computed by a unit-tested function. Show Figure 1 and the receipt.

Q2. What is your accuracy?

A. Macro-F1 0.8x on our own 200 labeled Indian narrations - there is no public benchmark for this data (verified), so we report exactly what we can defend, with n. Never quote a number you cannot reproduce.

Q3. Isn't a bootstrap overkill for a hackathon?

A. It is 15 lines, runs in milliseconds, and it is the only method that is honest on 3-8 months of messy flows - it borrows the user's own history instead of assuming a distribution. Kunsch 1989; seed fixed for reproducibility.

Q4. How is this different from Excel?

A. Excel will not catch a failed UPI reversal, ask before deleting a duplicate, or refuse to answer on 2.2 months of data. The intelligence is in the ledger hygiene and the honesty rules, not the arithmetic.

Q5. What about privacy / DPDP?

A. Everything stays local: SQLite on the user's machine. The LLM sees only redacted merchant substrings and amount bands - never account numbers or names - and there is a documented Ollama fallback that removes the cloud entirely.

Q6. What if a statement PDF fails?

A. It fails loudly and honestly: password-protected and scanned PDFs return clear messages and the CSV path. We chose visible failure over silent wrong parses - a wrong ledger poisons every downstream number.

Q7. Why not just auto-delete duplicates?

A. Overlapping exports make duplicates possible, not certain - Fellegi-Sunter's P class exists exactly for this. We flag and ask; silent deletion is invisible corruption of the forecast.

Q8. How reliable is the forecast?

A. We backtest it: rolling-origin evaluation (Hyndman fpp3) reports band coverage and P50 error on held-out months. The band is the promise: on thin data it widens, and the tool says so instead of bluffing.

Q9. What did you not build?

A. Multi-user tenancy, OCR for scanned PDFs, Account Aggregator integration, Isolation Forest anomalies, a what-if simulator. Each is scoped on the slide with the reason we cut it - restraint is a feature.

Q10. How does this scale to real users?

A. Per-bank adapters are config, not code; the LLM contract is one JSON schema; the math engine has no cloud dependency. The same architecture runs for 10 users or 10,000 - the ledger is per-user SQLite today, Postgres tomorrow.

Verified sources (all opened and checked during research)

Gao, L. et al. (2023). PAL: Program-aided Language Models. ICLR 2023. arXiv:2211.10435. https://arxiv.org/abs/2211.10435

Dziri, N. et al. (2023). Faith and Fate: Limits of Transformers on Compositionality. NeurIPS 2023. arXiv:2305.18654. https://arxiv.org/abs/2305.18654

Kunsch, H. R. (1989). The Jackknife and the Bootstrap for General Stationary Observations. The Annals of Statistics, 17(3), 1217-1241.

Fellegi, I. P., & Sunter, A. B. (1969). A Theory for Record Linkage. Journal of the American Statistical Association, 64(328), 1183-1210.

Iglewicz, B., & Hoaglin, D. C. (1993). How to Detect and Handle Outliers. ASQC Quality Press.

Leys, C. et al. (2013). Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median. Journal of Experimental Social Psychology, 49(4), 764-766.

Angelopoulos, A. N., & Bates, S. (2023). Conformal Prediction: A Gentle Introduction. Foundations and Trends in Machine Learning. arXiv:2107.07511. https://arxiv.org/abs/2107.07511

Goldstein, M., & Uchida, S. (2016). A Comparative Evaluation of Unsupervised Anomaly Detection Algorithms for Multivariate Data. PLoS ONE 11(4): e0152173.

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. IEEE ICDM 2008.

Hyndman, R. J., & Athanasopoulos, G. Forecasting: Principles and Practice (3rd ed.). OTexts. https://otexts.com/fpp3/

Gemini API - Structured Output (JSON mode). Google AI docs. https://ai.google.dev/gemini-api/docs/structured-output

Next.js Documentation. Vercel. https://nextjs.org/docs

FastAPI Documentation. https://fastapi.tiangolo.com/

pdfplumber. GitHub. https://github.com/jsvine/pdfplumber

SQLite Documentation. https://sqlite.org/docs.html